

第13-14课 PDF解析整理版

来源：飞书「ChatBI 项目系统蓝图」解析整理版。原课件 PDF 附件无下载权限，本文为文档正文导出。

第 13-14课 PDF 解析整理版

第 13 课+第 14 课： 问题剖析+原理落地

[要点] 两课核心关系

第13课与第14课构成了ChatBI从"Prompt 全量硬编码 Schema"到"按需动态召回 Schema"的关键第一公里：

- 第13课（问题剖析）：明确告诉系统"为什么全量 Schema 注入不可持续"——通过三个真实企业级歧义案例（欧洲市场销售额选错字段 / 各产品线成本字段歧义 / 利润按客户类型拆解涉及 5 表 Join），把问题拆解为 Schema Linking 的三项子任务（表召回、字段匹配、Join 路径推理），并指出"语义匹配"是把文本变成向量、用数学距离衡量相关性的解题方向。
- 第14课（原理落地）：紧接第13课结尾"把文本变成向量，用距离衡量相关性"这一句话，把方向拆成三个动作（文本→向量、向量→存储、距离→检索），用约 80 行代码手写一个最简向量检索系统，只用 numpy 就能在表级召回场景下跑通。

[链接] 逻辑衔接点

- 方向继承：第14课开场直接引用第13课小结里"把文本变成向量，用距离衡量相关性"作为开场提问，并把它拆成三个具体动作（谁来变 / 存在哪 / 怎么算）。
- 场景复用：第14课用到的演示表（dim_customers / sales_orders / dim_products / exchange_rates / finance_expenses）和演示问题（"欧洲市场最近三个月的销售额" / "各产品线的毛利率" / "上个月研发费用和销售费用对比"），正是第13课案例 1（欧洲市场销售额选错表/字段）和案例 2（各产品线成本字段歧义）的同一批素材——前者把"为什么需要表召回"讲透，后者把"表召回怎么做"落地。
- 能力扩展：第14课把第13课三项子任务里的"表召回"做了完整 demo，但不处理"字段匹配"和"Join 路径推理"（这两块留到第 16-17 课）；同时把"为什么需要工程化"的四个缺陷（无持久化、暴力 O(n)、无元数据过滤、无批量操作）显式列出，作为第 15 课 LangChain + ChromaDB 的引入动机。
- 接口契约预留：第14课的 search_tables(query, index, client, top_k=3) 暴露了"问题 → Top-K 表名 + 相似度分数"的接口形态，这正是第18课 Schema Linking Pipeline 里"表召回子模块"要对接的契约。

两节课的关系 / 总结

维度

第 13 课 (Schema Linking 问题剖析)

第 14 课 (Embedding 相似度检索)

所处阶段

第四阶段《让系统"理解业务"——Schema Linking + RAG》第一节

第四阶段《让系统"理解业务"——Schema Linking + RAG》第二节

核心问题

5 张表的全量硬编码 Schema 撑不到企业真实规模

语义匹配的方向太抽象, 需要从 0 到 1 跑通一遍

关键瓶颈

Token 窗口爆炸 / 噪声淹没 / 歧义字段干扰决策

无持久化 / 暴力 $O(n)$ / 无元数据过滤 / 无批量操作

核心概念

Schema Linking 三项子任务 / 语义匹配

Embedding / 余弦相似度 / 内存索引

关键产出

三个真实案例 + 子任务框架 + 解法方向

`cosine_similarity` / `build_table_index` / `search_tables`

在链路中的位置

业务问题定义层

表级召回子模块的"从 0 到 1"原型

下一课接力

拆解为三项子任务, 引出"语义匹配"方向

暴露 4 个工程缺陷，引出第 15 课 LangChain + ChromaDB

一句话关系总结：

- 第 13 课让系统"知道问题有多难"（从 5 张表到 100 张表的全量 Schema 注入瓶颈 + 三个真实案例 + 三项子任务框架）；
- 第 14 课让系统"先跑通一个子任务"（用 80 行代码把"表召回"从概念落到 demo，证明向量检索的核心逻辑不复杂，复杂的是工程化）；
- 衔接关系：第 14 课用第 13 课的同一批演示表和问题做落地，并在结尾把"工程化能力"作为下一课接力点；为第 18 课 Schema Linking Pipeline 预留了 search_tables 这一表召回子模块的接口契约。

第 13 课：企业数据为什么难用？Schema Linking 问题剖析

课程定位与目标

课程定位：

- 所处阶段：第四阶段《让系统"理解业务"——Schema Linking + RAG》第一节
- 本课核心问题：第 5-12 课的系统能够"从自然语言到 SQL 再到结果返回"，但 Prompt 中 Schema 信息的注入方式是全量硬编码。当企业数据规模扩大到几十张表、上百字段时，这种方式会导致上下文窗口爆炸、噪声淹没关键信息、模型注意力分散。如何让系统只看到"与当前问题相关的数据"，是第四阶段要解决的核心问题。

本节课目标：

- 理解"全量 Schema 注入"在企业规模下为什么不可持续
- 通过真实歧义案例，建立对 Schema Linking 三项子任务（表召回、字段匹配、Join 路径推理）的认知框架
- 理解"语义匹配"的基本思路——把文本变成向量，用距离衡量相关性
- 为第 14-17 课的具体技术实现建立问题意识

开场引入

来看当前系统 prompt_builder.py 中的一段代码：

```
SCHEMA = ""
表: dim_customers (客户维度表)
- customer_id INT 主键
```

```
- customer_name VARCHAR(100) 客户名称
- customer_type VARCHAR(50) 客户类型: OEM整车厂 / 储能集成商 / ...
- industry VARCHAR(50) 客户行业: 交通 / 能源 / 工业 / 特种交通
- country VARCHAR(50) 具体国家, 如 Germany
- region VARCHAR(50) 大区, 如 欧洲、北美

表: dim_products (产品维度表)
- product_id INT 主键
- product_name VARCHAR(100) 产品名称
...
(省略, 实际约 60 行)
"""
```

这段 Schema 定义大约占 1200 个 Token。对于当前 5 张表的场景，勉强能放进 Prompt，模型也还能注意到关键字段。

但如果面对一个真实企业——

[要点] 真实企业的数据库通常有 30-100 张表、500-2000 个字段。如果把全量 Schema 塞进 Prompt，Token 消耗可能达到 10000-30000，轻则占满上下文窗口让模型无法输出足够长的 SQL，重则让模型在“干草堆里找针”时频繁出错。

这就是第四阶段要解决的问题。

一、全量 Schema 注入的三个瓶颈

1. Token 窗口爆炸

以当前 5 张表为例，Schema 部分约 1200 Token。假设企业扩展到 50 张表（这在制造业、零售业非常常见），Schema 部分将膨胀到约 12000 Token。

再加上 Few-shot 示例、RULES、ERROR_GUARDS、指标知识……一个完整的 Prompt 可能超过 20000 Token，已经逼近甚至超过大模型的高效注意力范围。

2. 噪声淹没关键信息

即使模型的上下文窗口足够大（比如 128K），把全量信息塞进去仍然有问题——注意力稀释。

一个直觉的类比：如果考试只考第 3 章，但老师把整本书复印给你并说“答案在里面”，找到关键信息的效率会显著低于只给第 3 章的情况——大模型的注意力机制也类似，无关信息越多，聚焦关键信息的能力越弱。

3. 歧义字段干扰决策

当 Schema 中存在大量语义相近的字段时，模型容易混淆。比如当前 5 张表中已经存在：

- gross_amount（含税总额）vs net_amount（不含税收入）
- standard_cost（标准成本）vs material_cost（材料成本）vs labor_cost（人工成本）
- region（大区）vs country（具体国家）
- selling_expense（销售费用总项）vs marketing_expense（市场费用，属于销售费用子项）

这些歧义在 5 张表时通过 RULES 和 ERROR_GUARDS 尚能兜住。但当表数量增长 10 倍，这类歧义字段的组合数呈指数增长，规则方式维护变得逐渐吃力。

二、真实歧义案例

以下三个案例来自第 7-8 课的评估结果和真实企业项目经验，它们分别对应 Schema Linking 的三项子任务。

案例 1："欧洲市场的销售额"——选错表、选错字段

用户问题：欧洲市场最近三个月的销售额是多少？

模型可能的错误：

```
SELECT SUM(gross_amount) FROM sales_orders WHERE region = 'Europe'
```

问题分析：

- gross_amount 是含税总额，财务口径应该用 net_amount——字段选错
- region = 'Europe': sales_orders 表确实有 region 字段，但里面存的是什么值？dim_customers 表也有 region 字段。两个 region 是同一回事吗？——字段归属歧义
- 没有关联 exchange_rates 做汇率转换——表遗漏

案例 2："各产品线的成本"——字段歧义

用户问题：查询各产品线的成本

模型可能的错误：

```
SELECT product_line, SUM(standard_cost) FROM dim_products GROUP BY product_line
```

问题分析：

dim_products 表中有三个成本字段：

字段

含义

适用场景

standard_cost

标准成本（含分摊制造费用）

预算对比、差异分析

material_cost

材料成本

产品成本结构拆解

labor_cost

人工成本

产品成本结构拆解

"各产品线的成本"在业务语境中通常指实际销售成本 = (material_cost + labor_cost) * quantity，需要关联 sales_orders 表获取数量。模型如果看到三个成本字段且没有额外指引，很容易选错。

案例 3："利润按客户类型拆解"——Join 路径复杂

用户问题：按客户类型看上一个月的利润

正确的查询需要：

- sales_orders：获取收入（net_amount）和数量（quantity）
- dim_products：获取单位成本（material_cost + labor_cost）
- dim_customers：获取客户类型（customer_type）
- exchange_rates：汇率转换
- finance_expenses：期间费用

这涉及 5 张表的 Join，且 Join 路径、Join 条件各不相同。模型如果面对全量 Schema，需要自己推断所有的关联关系——这正是 SQL 生成错误率最高的环节。

三、Schema Linking 的定义与三项子任务

[要点] Schema Linking（模式链接）的本质是：给定一个用户问题，从数据库全量 Schema 中精准定位出"回答这个问题所需的最小信息集"——具体包括哪些表、哪些字段、以及表之间如何关联。

这可以拆解为三项子任务：

子任务 1：表召回 (Table Retrieval)

输入：用户问题 + 全量表列表

输出：与问题相关的 Top-K 张表

核心挑战：用户说的是自然语言（"欧洲市场的销售额"），而表名是技术命名（sales_orders、dim_customers）。如何判断一个自然语言问题"需要"哪些表？

子任务 2：字段匹配 (Field Matching)

输入：用户问题 + 候选表的所有字段

输出：与问题最相关的字段子集

核心挑战：同一张表内多个字段语义接近（如 gross_amount vs net_amount），如何选出正确的那个？

子任务 3：Join 路径推理 (Join Resolution)

输入：一组需要关联的表

输出：合法的 Join 路径和 Join 条件

核心挑战：两张表之间可能存在多条关联路径（直接外键 vs 通过中间表），如何选择最合理的路径？

四、为什么"语义匹配"是解法

传统方案会尝试用规则解决——比如维护一个映射表：

"欧洲" → region 字段
"销售额" → net_amount 字段
"产品线" → dim_products 表

这种方式在小规模下可行，但有明显的局限性：

- 覆盖不全：用户可能说"营收""收入""销售额""GMV"——全是同一个意思，但规则需要逐一枚举
- 不可扩展：每新增一张表、一个字段，都需要人工维护映射规则
-

无法处理模糊表达："上个季度各条线赚了多少钱"——这里"条线"指产品线还是业务线？"赚了"指收入还是利润？

语义匹配的思路是把文本变成向量，用数学距离衡量相关性：

"欧洲市场的销售额" → [0.12, -0.34, 0.87, ...] （1536维向量）

```
"dim_customers: 客户维度表, 含客户类型、国家、大区" → [0.08, -0.29, 0.82, ...]
"sales_orders: 销售订单表, 含收入、数量、日期" → [0.15, -0.31, 0.85, ...]
"finance_expenses: 费用表, 含四大费用" → [-0.21, 0.45, 0.11, ...]
```

计算"用户问题向量"与"每张表描述向量"的余弦相似度，分数最高的就是最相关的表。

[要点]

这种方法的核心优势：不需要人工维护映射规则，模型自动通过语义理解判断相关性。新增表时只需为其生成向量描述，无需修改任何代码逻辑。

五、Schema Linking 在系统中的位置

当前系统的 Prompt 组装流程（第 6 课定义）：

1. RULES
2. SCHEMA
3. FEW_SHOT_EXAMPLES
4. 用户问题

引入 Schema Linking 后的流程：

1. RULES
2. 动态检索出来的精简 SCHEMA ← 变化只在这一环
3. FEW_SHOT_EXAMPLES
4. 用户问题

变化只在 Schema 来源这一环：从"硬编码全量"变成"动态精简"。其余模块（RULES、Few-shot、指标知识、LLM 调用、数据库执行）全部保持不变。

六、第四阶段的整体路线图

第四阶段共 8 节课，围绕 Schema Linking 和 RAG 两条主线展开：

- 第 14 课是 Embedding 技术的入门课——用 60 行代码手写一个最简向量检索，直面原理
- 第 15 课是工程化升级——引入 LangChain + ChromaDB，建立后续所有课复用的基础设施
- 第 16-17 课在基础设施之上，分别实现字段匹配和 Join 推理两个子模块
- 第 18 课将三个子模块串联为完整 Schema Linking Pipeline
- 第 19-20 课引入指标 RAG，最终将 Schema Linking + RAG 联合整合到系统中

小结

本节课明确了三个核心认知：

1. 全量 Schema 注入不可持续——Token 爆炸、噪声干扰、歧义字段是三大瓶颈
2. Schema Linking 分三步走——表召回、字段匹配、Join 路径推理，各自解决不同层面的问题
3. 语义匹配是解法方向——把文本变成向量，用数学距离代替人工规则

下一课将从零实现一个最简的向量检索

demo，让“把文本变成向量、用距离衡量相关性”这个概念落地为可运行的代码。

第 14 课：向量检索基础——从零手写 Embedding 相似度检索

课程定位与目标

课程定位：

- 所处阶段：第四阶段《让系统“理解业务”——Schema Linking + RAG》第二节

- 本课核心问题：第

13

课提出了“语义匹配”的解题方向——把文本变成向量，用距离衡量相关性。但这到底是什么意思？向量从哪来？距离怎么算？本课用约 80 行代码手写一个最简向量检索系统，让原理完全可见。

本节课目标：

- 理解 Embedding 的直觉含义——文本到坐标的映射，语义相近则坐标相近
- 理解余弦相似度的物理含义和计算方式
- 能从零实现“表描述向量化 → 内存存储 → 用户问题检索”的完整闭环
- 验证向量检索在“表级召回”场景下的实际效果

开场引入

第 13 课结尾提到：

把文本变成向量，用距离衡量相关性。

这句话包含三个关键动作：

1. 把文本变成向量——谁来变？怎么变？

2. 存储向量——存在哪里？什么格式？
3. 计算距离——用什么公式？阈值怎么定？

本课用一个完整的可运行脚本回答这三个问题。这个脚本最终能实现：输入一个自然语言问题，输出"与这个问题最相关的 Top-K 张表"。

一、什么是 Embedding

1. 直觉理解

Embedding（向量嵌入）是一种将文本映射为固定长度浮点数数组的技术。

一个类比：城市坐标。北京 (39.9, 116.4)，上海 (31.2, 121.5)，纽约 (40.7, -74.0)——坐标本身是数字，但从数字中可以计算出"北京和上海距离近，北京和纽约距离远"。

Embedding 做的事情类似：给每段文本分配一个"坐标"（只不过维度从 2 变成了 1536），然后通过坐标间的距离判断语义相关性。

2. 调用 Embedding API

Embedding 调用与 LLM 调用使用同一套 OpenAI 兼容接口：

```
def get_embedding(self, text: str) -> list[float]:
    resp = self.client.embeddings.create(
        model=self.embedding_model,
        input=text,
    )
    embedding = resp.data[0].embedding
    return embedding
```

调用一次，返回一个浮点数列表：

```
from llm_client import LLMClient

client = LLMClient()
vec = client.get_embedding("欧洲市场的销售额")
print(f"向量维度: {len(vec)}")    # 输出: 向量维度: 1536
print(f"前5个值: {vec[:5]}")     # 输出: [-0.0123, 0.0456, -0.0089, ...]
```

[要点]

Embedding

模型的输入是任意长度的文本，输出是固定维度的浮点数数组。语义相近的文本，其输出向量在高维空间中的"方向"更接近。

3. 模型选型说明

本课程使用 OpenAI 兼容接口调用 Embedding 模型，具体模型通过 .env 中的 EMBEDDING_MODEL 配置：

模型

维度

特点

text-embedding-3-small

1536

性价比高，适合小规模系统

text-embedding-3-large

3072

精度更高（评估两句话之间差异的精度越高），Token 消耗相同但向量更大

生产环境中如果数据不能外传，可以替换为本地 Embedding 模型（如 bge-small-zh-v1.5），只需修改 .env 相关配置，代码逻辑无需变动。

二、余弦相似度

1. 为什么用余弦而非欧氏距离

Embedding

向量的方向比长度更能反映语义。两段语义相近的文本，其向量可能因文本长度不同导致向量模长不同，但方向（夹角）仍然接近。

余弦相似度正是只关注方向、忽略长度的度量方式。

2. 计算公式

- 分子：两个向量的点积（对应元素相乘再求和，3072维，每个维度分别相乘）
- 分母：两个向量的模长之积（归一化，每个维度的值平方、相加、开根号）
- 值域：[-1, 1]，1 表示方向完全相同，0 表示正交无关，-1 表示方向完全相反 在实际 Embedding 检索中，相关文本的余弦相似度通常落在 0.3~0.9 之间。。

3. 代码实现

```
import numpy as np

def cosine_similarity(vec_a: list[float], vec_b: list[float]) -> float:
    a = np.array(vec_a)
    b = np.array(vec_b)
    dot_product = np.dot(a, b)
    norm_a = np.linalg.norm(a)
    norm_b = np.linalg.norm(b)
    if norm_a == 0 or norm_b == 0:
        return 0.0
    return float(dot_product / (norm_a * norm_b))
```

只用了 numpy 的点积和范数计算，没有任何黑盒。

三、手写最简向量检索系统

1. 整体思路

两阶段：

- 离线索引：把每张表的描述文本转为向量，存入内存字典
- 在线检索：用户问题转为向量，与所有表向量逐一计算余弦相似度，取分数最高的 Top-K

2. 表描述文本构造

向量检索的效果直接取决于“被检索的文本”写得好不好。每张表的描述需要包含：

- 表的业务用途（不只是技术字段列表）
- 核心字段的业务含义
- 典型的分析场景

```
TABLE_DESCRIPTIONS = {
    "dim_customers": (
        "客户维度表：存储客户基本信息，包括客户名称、客户类型"
        "（OEM整车厂、储能集成商、电网集团等）、所属行业（交通、能源、工业）、"
        "所在国家和销售大区（欧洲、北美、亚太等）。"
        "用于按客户维度分析收入、利润和订单分布。"
    ),
    "sales_orders": (
        "销售订单表：记录每笔销售订单的详细信息，包括订单日期、订单状态、"
        "数量、单价、折扣、含税总额（gross_amount）、不含税收入（net_amount）、币种。"
        "通过 customer_id 和 product_id 关联客户表和产品表。"
        "是收入分析、订单统计的核心事实表。"
    ),
    # ... 其余表类似
}
```

[要点]

表描述的质量决定了检索效果的上限。描述中既要有技术信息（字段名、类型），也要有业务信息（适用场景、分析维度），这样才能同时匹配"技术型提问"和"业务型提问"。

3. 构建索引

```
def build_table_index(client: LLMClient) -> dict[str, list[float]]:
    """将所有表的描述文本转为 Embedding 向量，构建内存索引"""
    index = {}
    for table_name, description in TABLE_DESCRIPTIONS.items():
        embedding = client.get_embedding(description)
        index[table_name] = embedding
    return index
```

执行后，index 是一个字典：key 是表名，value 是多维浮点数列表。这里的整个"向量数据库"就是一个 Python dict。

4. 执行检索

```
def search_tables(query, index, client, top_k=3):
    """根据用户问题检索最相关的表"""
    # 将用户问题转为向量
    query_embedding = client.get_embedding(query)

    # 计算与每张表的相似度
    scores = []
    for table_name, table_embedding in index.items():
        score = cosine_similarity(query_embedding, table_embedding)
        scores.append((table_name, score))

    # 按相似度降序排序，取 Top-K
    scores.sort(key=lambda x: x[1], reverse=True)
    return scores[:top_k]
```

逻辑极其简单：遍历所有表向量，逐一计算余弦相似度，排序取前 K 个。

四、运行验证

环境准备

本课新增依赖 numpy:

```
uv add numpy
```

运行演示脚本

```
uv run vector_search_demo.py
```

预期输出示例

```
正在构建表描述向量索引...
[OK] dim_customers      → 向量维度 1536
[OK] dim_products       → 向量维度 1536
[OK] sales_orders       → 向量维度 1536
[OK] exchange_rates     → 向量维度 1536
[OK] finance_expenses   → 向量维度 1536
.....
索引构建完成，共 9 张表

=====

向量检索演示：根据用户问题召回相关表
=====

问题：欧洲市场最近三个月的销售额是多少？
召回结果：
[OK] sales_orders       相似度：0.5832
[OK] dim_customers      相似度：0.5216
[OK] exchange_rates     相似度：0.4371

问题：各产品线的毛利率
召回结果：
[OK] dim_products       相似度：0.5944
[OK] sales_orders       相似度：0.5127
  • finance_expenses    相似度：0.3685

问题：上个月的研发费用和销售费用对比
召回结果：
[OK] finance_expenses   相似度：0.6721
  • sales_orders        相似度：0.3214
  • dim_products         相似度：0.2856
```

效果分析

从输出可以看到：

- "欧洲市场的销售额"——召回了 sales_orders（核心事实表）、dim_customers（因为"欧洲"是客户维度）、exchange_rates（多币种需要汇率），三张都是正确的
- "各产品线的毛利率"——召回了 dim_products（产品线信息）和 sales_orders（收入数据），但没有召回 finance_expenses（因为毛利不涉及期间费用），合理
- "研发费用和销售费用"——精准命中 finance_expenses，其余表相似度较低

[要点] 仅用 80 行代码和零外部依赖（numpy 除外），就实现了一个能正确工作的表级召回系统。这证明了向量检索的核心逻辑并不复杂——复杂的是工程化（持久化、大规模、可维护）。

五、局限性分析

这个手写方案虽然能 work，但有四个明显的工程缺陷：

缺陷

说明

后果

无持久化

索引存在内存 dict 中，进程退出即丢失

每次启动都要重新调 Embedding API（费时费钱）

暴力搜索 $O(n)$

遍历所有向量计算相似度

几张表没问题，500 张表会明显变慢

无元数据过滤

无法按"主题域""表类型"预筛选

无法在检索前排除明显无关的表

无批量操作

每条文本单独调 API

构建索引时网络开销大

这些缺陷在当前几张表的场景下不影响使用，但当数据规模增长到几十张表时就会成为瓶颈。

下一课将引入 LangChain + ChromaDB 来解决这些问题：

- ChromaDB 提供持久化存储和 ANN（近似最近邻）高效搜索
- LangChain 提供统一抽象层，让 Embedding 模型和向量数据库可替换

核心算法和思路不变——仍然是"文本转向量 → 存储 → 余弦检索"——但工程能力会得到质的提升。

小结

本节课完成了向量检索的"从 0 到 1"：

拓展思考：

- 现在索引存在内存 dict 中，进程退出即丢失，有什么快速的方法可以解决此问题

附件（飞书无下载权限）：第 13 课：企业数据为什么难用？Schema Linking 问题剖析.pdf

附件（飞书无下载权限）：第 14 课：向量检索基础——从零手写 Embedding 相似度检索.pdf